

Classes and Objects

中央資訊工程系 鄭永斌副教授

1

Classes and Objects

- **What is a class ?**
 - class is just a type
 - When you define a class X , you pass another X.h to be included by others.
 - Without X.obj, it is still possible for others to compile his module.
 - In C++, a class is just like a primitive type INT, CHAR
- **What is an object?**
 - When you use a class to create a variable, the variable (along with its allocated memory) is called an object
 - A class can have many objects
 - Each object has its own memory
 - Object is simply memory+behaviors (code)

```
class Point {
    int z ;
public:
    int x ;
    int y ;
};

Point p,p1 ;
main() {
    p.x = 100;
    p.y = 200 ;
}
```

2

An example

```
in Dog.h
#ifndef _DOG
#define _DOG

class Dog {
public:
    void setAge(int age);
    int getAge();
    void setWeight(int weight);
    int getWeight();
    void speak();
private:
    int age;
    int weight;
};

#endif

#include "Dog.h"
void Dog::setAge(int age)
{
    this->age = age;
}

int Dog::getAge()
{
    return age;
}

void Dog::setWeight(int weight)
{
    this->weight = weight;
}

int Dog::getWeight()
{
    return weight;
}

void Dog::speak()
{
    cout << "BARK!!" << endl;
}
```

3

Constructor & Destructor

In C, 當你使用 **struct** 時，你只有定義了資料，有時候你還得自己動手做一些初始化的動作

例如

```
typedef struct dog_body {
    int age;
    int weight;
    char *name;
} Dog;
```

- 假設你事先不能知道 **name** 的長度，你必須自己想辦法配置空間與初始化

```
Dog mydog;

// somehow, you got a dog name from input, it is stored in dname
mydog.name = malloc(strlen(dname));
strcpy(mydog.name, dname);
```

- 就算別人幫你定義好 Dog 的資料結構，很多事情你得自己來，或者是看書定義資料結構的人好心幫你多寫許多的與 Dog 有關的 function，但是還是得告訴你要去呼叫
- 我們希望這些程式能夠在初始化自動幫我們事先完成

4

Constructor and Destructor

```
class Dog {
public:
    Dog();//Constructor
    ~Dog();//Destructor
    void setAge(int age);
    int getAge();
    void setWeight(int weight);
    int getWeight();
    void speak();
private:
    int age;
    int weight;
};

Dog::Dog()
{
    age = 0;
    weight = 0;
    cout << "Dog Constructor Called" << endl;
}

Dog::~Dog()
{
    cout << "Dog Destructor Called" << endl;
}
```

5

Using a class (instantiation)

Let's try it

```
#include "car.h"
#include <iostream>
using namespace std ;
Car::Car() { cout << "Car is constructed" << endl ; }
Car::~Car() { cout << "Car is destroyed!" << endl ; }

//-----
#include "car.h"
Car honda ;
main() {
    Car camery ;
    honda.color = 1 ;
    camery.color = 100 ;
}
```

Make it a global variable

Make it a local variable

Output

```
Car is constructed!
Car is constructed!
Car is destroyed!
Car is destroyed!
```

6



- So, when is Honda's destructor is called
- Ans:
 - after the main program
 - There is a piece of code after main() to handle the globally allocated objects

- A global variable is allocated at Data Segment
 - It disappears (**deallocated, destroyed**) at the exit of program
- A local variable is allocated at Stack
 - It disappears at the end of functions (**main()** is actually a function)

```
#include "car.h"
#include <iostream>
using namespace std ;

Car::Car() { cout << "Car is
constructed" << endl ; }

Car::~Car() { cout << "Car is
destroyed!" << endl;}

//-----
#include "car.h"
Car honda ;
main() {
    Car *camery = new Car();
    honda.color = 1 ;
    camery->color = 100 ;
}
```



```

004010c9 <main+121>:  mov     0xffffffff(%ebp),%edx
004010cc <main+124>:  mov     %edx,0xffffffff(%ebp)
004010cf <main+127>:  jmp     0x40109 <main+185>
004010d1 <main+129>:  lea     0xc(%ebp),%ebp
004010d4 <main+132>:  mov     0xffffffff(%ebp),%eax
004010d7 <main+135>:  mov     %eax,0xffffffff(%ebp)
004010da <main+138>:  mov     0xffffffff(%ebp),%edx
004010dd <main+141>:  mov     %edx,0xffffffff(%ebp)
004010e0 <main+144>:  cmpb    $0x0,0xffffffff(%ebp)
004010e4 <main+148>:  je      0x4010f1 <main+161>
004010e6 <main+150>:  mov     0xffffffff(%ebp),%eax
004010e9 <main+153>:  mov     %eax,(%esp,1)
004010ec <main+156>:  call    0x401f10 <_ZdlPv>
004010f1 <main+161>:  mov     0xffffffff(%ebp),%edx
004010f4 <main+164>:  mov     %edx,0xffffffff(%ebp)
004010f7 <main+167>:  mov     0xffffffff(%ebp),%eax
004010fa <main+170>:  mov     %eax,(%esp,1)
004010fd <main+173>:  movl    $0xffffffff,0xffffffff(%ebp)
00401104 <main+180>:  call    0x40d050 <_Unwind_Sjlj_Resume>
00401109 <main+185>:  movl    $0x1,0x404010
00401113 <main+195>:  mov     0xffffffff(%ebp),%eax
00401116 <main+198>:  movl    $0x6a,(%eax)
0040111c <main+204>:  movl    $0x0,0xffffffff(%ebp)
00401123 <main+211>:  lea     0xffffffff(%ebp),%eax
00401126 <main+214>:  mov     %eax,(%esp,1)
00401129 <main+217>:  call    0x40e20 <_Unwind_Sjlj_Unregister>
0040112e <main+222>:  mov     0xffffffff(%ebp),%eax
00401131 <main+225>:  lea     0xffffffff(%ebp),%esp
00401134 <main+228>:  pop     %ebx
00401135 <main+229>:  pop     %esi
00401136 <main+230>:  pop     %edi
00401137 <main+231>:  pop     %ebp
00401138 <main+232>:  ret

```

這一段指令都在做
camery = new Car();

13

What is heap?(自由區 · free store)

- Heap in a process's image (let's rock)
- In C,
 - use *malloc*, *calloc*, *free*
- In C++,
 - use *new*, *delete*
- Be aware of **memory leaking**

14

Summary

- Just like C, you have 3 places to place your object
 - data (life time goes with program)
 - stack (life time goes with functions)
 - heap (life time vary)

15

Exercise

- 下列的程式

```

Car * foo() {
    Car mazda ;

    .....

    return &mazda ;
}

```

16

物件的陣列(Object Array in C++)

Table tbl[10] ;

- 如果一個物件可再不給訂初始參數的情況下被建立出來，就能定義由此class所組成的陣列。
- 他會建立一個內含10個 table 物件的陣列
- delete
 - delete[] tbl ;

17

Dynamic memory allocation

- So, now you know objects are created in stack, data, and heap (allocated by *new*)
- a new classification
 - statically allocated object
 - objects are of fixed, known size and the compiler arranges the required space as it turns source code into a binary or executable program. Statically allocated objects that are of local *scope* are put into a memory space known as the stack. Statically allocated objects of global scope live in the global address space. The key point is that for these objects their size is fixed at compile time.
 - dynamically allocated object
 - we don't know the size of an object until program execution. Examples of this are a buffer to hold a block of text of variable size, or an *array* with an undetermined number of elements

18

- `int *IDpt = new int;`
`float *theMoney = new float;`
`char *letter = new char;`
- The "new" operator returns the address to the start of the allocated block of memory. This address must be stored in a **pointer**. New allocates a block of space of the appropriate size in the heap for the object. Dynamically allocated memory is accessed indirectly via a pointer. It is possible for new to fail. This will be the case if no memory is available. In this case new throw a "bad_alloc" exception. Exception handling will be discussed in a latter lesson.
- `int *IDpt = new int;`
`*IDpt = 5;`
`int *IDpt = new int(5);` // Allocates an int object and initializes it to value 5.
`char *letter = new char('J');`

19

19

- Dynamically allocated objects introduce a new twist; they must be explicitly deleted when no longer needed by a program. This is done using the **"delete"** operator. Delete releases the memory used by the object. That memory is then available for reuse.

```
delete IDpt;
delete theMoney;
delete letter;
```

20

20

Dynamic allocated array

```
int *pt = new int[1024]; //allocates an array of 1024 ints
double *myBills = new double[10000];
```

```
int *pt = new int[1024]; //allocates an array of 1024 ints
int *pt = new int(1024); //allocates a single int with value 1024
```

- A dynamically allocated array is best initialized using a loop, as follows.

```
int *buff = new int[1024];
for (i = 0; i < 1024; i++)
{
    buff[i] = 52; //Assigns 52 to each element;
}
```

- free the dynamic allocated objects

```
delete[] pt;
delete[] myBills; //chapter11() will also release the memory :-)
```

21

21

Usage of Object array

```
#include <iostream>
#include <string>
using namespace std;

class Cat {
public:
    Cat(string name = "tom", string color =
        "black_and_white") : _name(name),
        color(color) {}
    ~Cat() {}
    void setName(string name) { _name =
        name;}
    string getName() {return _name;}
    void setColor(string color) { _color =
        color;}
    string getColor() {return _color;}
    void speak() {cout << "meow" << endl;}
private:
    string _name;
    string _color;
};

int main()
{
    // Single Argument form of array initialization
    list
    Cat myCats[4] =
    {"Homer", "Marge", "Bart", "Lisa"};

    // Explicit use of constructors in array
    initialization list
    Cat myOtherCats[3] = {
    Cat("Chris", "Gray"),
    Cat("Charles", "White"),
    Cat("Cindy", "BlueGray")
    };

    Cat *catpt = new Cat[27];
    Cat *pt;

    // Use accessor methods to initialize
    dynamically allocated class objects
    pt = catpt;
    for (int i = 0; i < 27; i++) {
        pt->setName("Felix"); // You can never have
        enough
        pt->setColor("Black"); // black cats named
        Felix
        pt++; // Can also increment pointer within the
        for statement
    }
```

22

22

Copy constructor and "=" operator

```
#include <iostream>
using namespace std;

class Employee {
public:
    Employee(char *name, int id);
    ~Employee();
    char *getName() {return _name;}
    //Other Accessor methods
private:
    int _id;
    char *_name;
};

Employee::Employee(char *name, int
id)
{
    _id = id;
    _name = new char(strlen(name) + 1);
    strcpy(_name, name);
}

Employee::~Employee()
{
    delete[] _name;
}

int main()
{
    Employee
    programmer("John", 22);
    cout << programmer.getName()
    << endl;
    return 0;
}
```

23

23

Comments

- Employee name is now stored in a dynamically allocated character array. It is of "string length + 1" to allow for the null terminator used with **C-style strings**, "\0".
- The **strcpy** function automatically adds the **null terminator** to the destination string.
- Also, notice that the destructor frees the memory used to hold the employee name. This is needed to avoid a memory leak,

24

24

```
int main()
{
    Employee programmer("John",22);
    cout << programmer.getName() << endl;

    //Lots of code ....

    Employee manager = programmer;
    //Creates a new Employee "manager",
    //which is an exact copy of the
    //Employee "programmer".

    delete programmer ;

    return 0;
}
```

- Let's key in the example and run it
- anything wrong?

25

25

26

26

Comments

- The above program contains a serious bug and may die with an **exception** when run
- The problem is that the default member-wise copy constructor is being used to create the "manager" object.
- But the member-wise copy copies the address stored in the pointer `_name` in "programmer" to the pointer `_name` in "manager".
- When you run the above program, programmer's destructor gets called first. It works fine and frees the block in memory. When manager's destructor is called, an exception is thrown when the delete is attempted because that block of memory has already been released.

27

27

```
class Employee {
public:
    Employee(char *name, int id);
    Employee(Employee &rhs);
    ~Employee();
    char *getName(){return _name;}
    int getId(){return _id;}
    //Other Accessor methods
private:
    int _id;
    char *_name;
};

Employee::Employee(char *name, int id)
{
    _id = id;

    _name = new char[strlen(name) + 1];
    //Allocates a character array object
    strcpy(_name, name);
}

Employee::~Employee() {
    delete[] _name;
}

Employee::Employee(Employee &rhs)
{
    _id = rhs.getId();
    _name = new char[strlen(rhs.getName()) + 1];
    strcpy(_name, rhs._name);
}
```

28

28

Let's see another example

```
#include <iostream>
using namespace std;

class Table {
    char *p;
    int sz;
public:
    Table(int s=15) {
        p = new char[sz];
        cout << "constructor" << endl;
    }
    ~Table() { delete[] p;
        cout << "destructor" << endl;
    }
};

void h() {
    Table t1;
    Table t2 = t1;
    Table t3;
    t3 = t2;
}

int main() { h(); }
```

- How many times the default constructor is called?
請問Table預設的建構式被呼叫了幾次?
- How many times the default destructor is called? (Table的解構式被呼叫了幾次?)
- What is the final output?

29

29

The Correct Way

```
class Table {
    Table(const Table&);
    Table& operator=(const Table&);
};

Table::Table(const Table& t) {
    p = new Name [sz=t.sz];
    for (int i = 0; i<sz; i++)
        p[i] = t.p[i];
}

Table& Table::operator=(const Table& t) {
    if (this != &t) {
        delete[] p;
        p = new Name [sz=t.sz];
        for (int i = 0; i<sz; i++) p[i] = t.p[i];
    }
    return *this;
}
```

絕大部分的情況下
「copy」operator 與
「=」operator 有很大的不同

根本的原因

copy operator 會初始化一塊未經
初始化的記憶體
而 = operator 必須妥善處理一個
已經建構好的物件

30

30